

Process Management

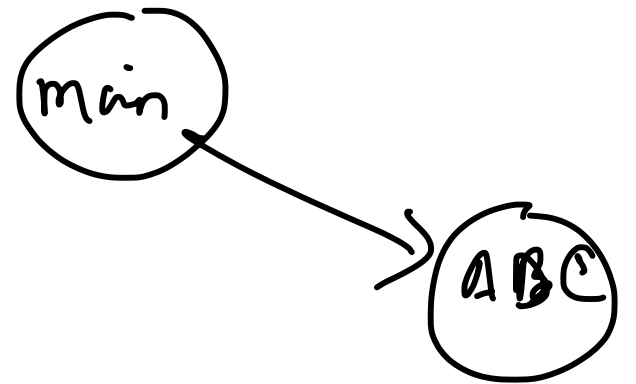
- 1) Process Data Structure and LifeCycle
- 2) Threads
- 3) Process Scheduling
- 4) Process Communication
 - Concepts of different techniques and mechanisms
 - POSIX-based primitives
 - (Signals)
- 5) Process Synchronization
 - Problem whenever concurrency
 - needs to be supported

→ Problem of race condition

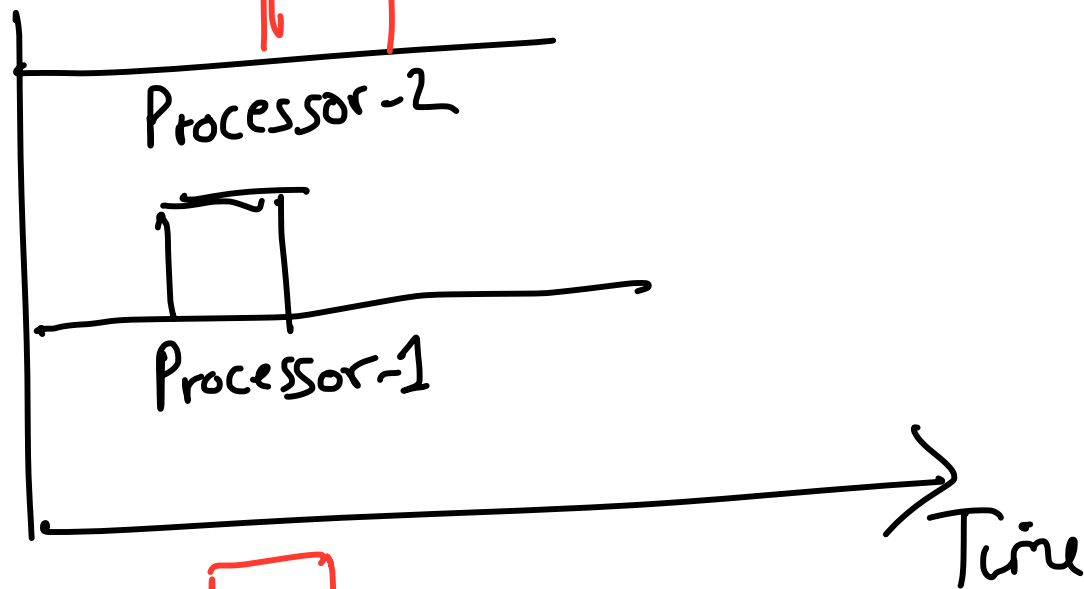
Concurrency

The support within ^{or across} 1 program to run two or more instructions at the same time.

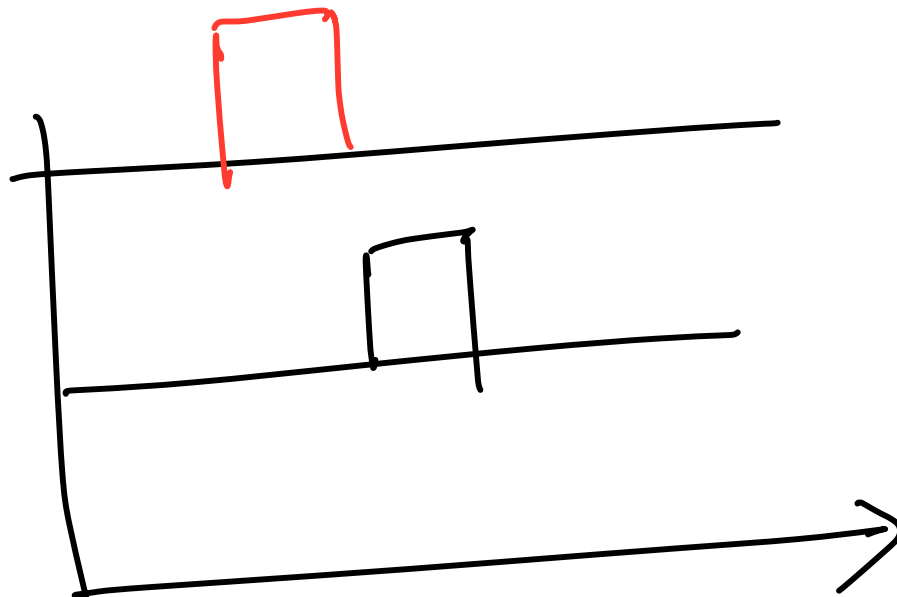
`pthread_create("ABC")`



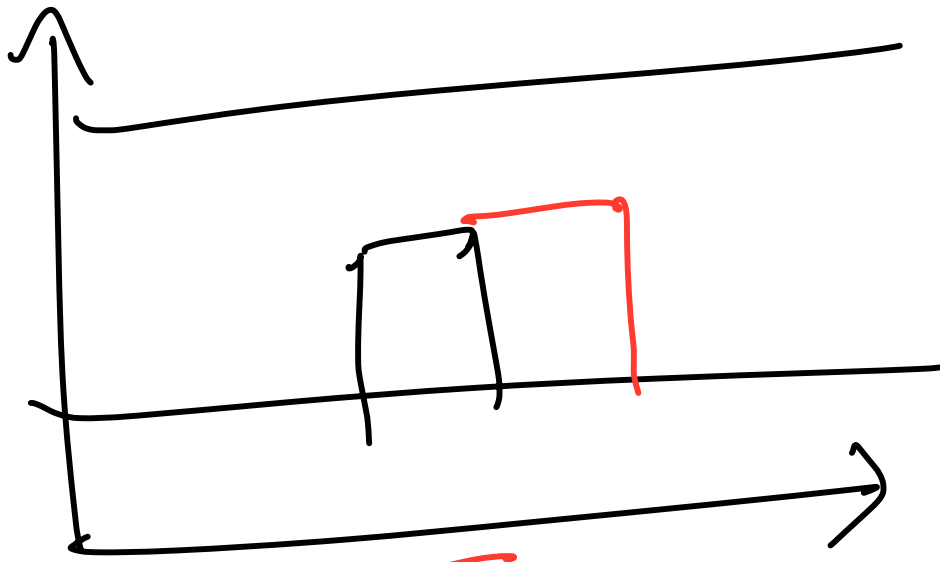
(a)



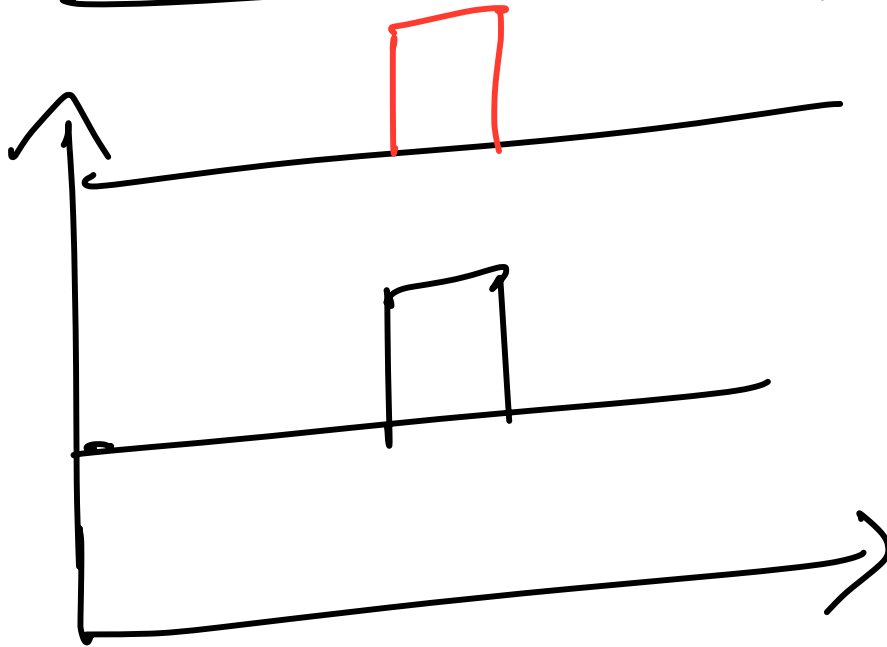
(b)



(c)



(d)



We say that there are no race conditions only if the sequence of execution does not affect the result.

If a program is run in parallel, then it must support concurrency. But supporting concurrency does not necessarily lead to parallel execution.

```
int a = 3;
```

```
void fun A ()  
{  
    Critical Section  
    {  
        a = a + 1;  
    }  
}
```

```
{  
    LD rax, a  
    ADD rax, #1  
    ST #1, rax  
}
```

```
void fun B ()  
{  
    Critical Section  
    {  
        a = a + 1;  
    }  
}
```

```
int main (void)  
{  
    tA = pthread_create (funA ...);  
    tB = pthread_create (funB ...);  
    pthread_join (tA);  
    pthread_join (tB);  
    printf ("%d", a);  
}
```

// Race condition → across threads, across processes, within the kernel's shared data

structures,

- (1) You need to have support for "mutual exclusion" $\rightarrow$ where one thread/process accesses a shared "resource", no other process should access it at the same time.
- (2) Support for bounded waiting $\rightarrow$ If two processes are competing for resources, then each process should get some chance of accessing the resource.
- (3) Avoiding deadlock is essential $\rightarrow$ If two or more processes wait for one another to get access to the resource, then none of them can progress. This situation is called deadlock, and such deadlocks need to be avoided.
- (4) No starvation $\rightarrow$ No process should be blocked if resource is available.

Is there any mechanism by which you can satisfy the four conditions?

The part of the process which accesses the shared resource is called "critical

Spin lock: $\rightarrow$ Use a shared variable called "lock" and check whether the resource is int a; available before accessing it.

```

int lock;
void funA ( ) {
    { while (lock == 1) ; }
    lock = 1;
    a = a + 1;
    lock = 0;
}

```

This solution fails to actually provide proper mutual exclusion. There is no software solution to the problem of satisfying all the above four conditions. But it is possible to set a "turn" variable to do "strict alternation", where thread A and thread B can alternate execution.

There is very high overhead of using spin locks.

Support for atomic operations $\rightarrow$ Provide some basic support for simple operations to be performed in an indivisible manner,

Most commonly there is a "Test-and-set lock" (TSL) instruction that is given.

```
while (TSL (&lock));
```

```
a = a + 1;
```

```
Reset Lock (&lock);
```

Does not solve the problem of busy-waiting/spinning.

Concept of semaphore variables → wait() and signal()

are two primitives. Before accessing the CS, you call wait, and while leaving the CS, you call signal. If the lock is set, the wait instruction sends the process to sleep. This process wakes up only when the signal instruction is executed. Avoids the requirement of busy waiting or spin locks.

```
Semaphore sem;  
void fnA() {  
    wait(&sem);  
    a = a + 1;  
    signal(&sem);  
}
```

```
void fnB() {  
    wait(&sem);  
    a = a + 1;  
    signal(&sem);  
}
```

```

int a, b;
a = 3; b = 4;
Semaphore semA, semB;

void funA() {
    wait(&semA);
    wait(&semB);
    a = a + b;
    b = b + 1;
    signal(&semA);
    signal(&semB);
}

void funB() {
    wait(&semB);
    wait(&semA);

    b = b + 1;
    a = a + b;
    signal(&semB);
    signal(&semA);
}

```

semA $\rightarrow$ locked (A)
 semB $\rightarrow$ locked (B)

Semaphores can lead to deadlocks, unless additional techniques are used to resolve / prevent them. One way of preventing such deadlocks is to impose a sequence of requesting for wait on shared variables / Semaphores -